

* Unit 3 Lecture*

✓ *This script will be a part of HW #2 Functions and Modules*

Slide 1: Introduction to Functions

```
print("Functions allow us to reuse code and make our programs more modular.")
```

Slide 2-3: Defining and Calling a Simple Function

```
def greet():  
    print("Hello! Welcome to Python Functions.")  
  
# Run the function  
greet()
```

Challenge 1

```
# Challenge 1: Define a function that prints "Python is fun!"  
# Complete the code below  
# def python_fun():  
#     ...
```

Slides 6-7: Function with Parameters

```
def personalized_greeting(name):  
    print(f"Hello, {name}! Welcome to Python Functions.")  
  
# Run the function  
personalized_greeting("Alice")
```

Challenge 2: Define a function that takes two arguments and prints their sum

```
# Complete the code below  
# def print_sum(a, b):  
#     ...
```

Slide 7: Returning Values from Functions

```
def add(a, b):  
    return a + b  
  
# Run the function and print the result  
result = add(3, 5)  
print(f"The sum is: {result}")
```

Challenge 3: Define a function that returns the product of two numbers

```
# Complete the code below  
# def multiply(a, b):  
#     ...
```

Slide 8-9: Local and Global Variables

```
# Example 4: Local Variable  
def local_variable_example():  
    local_var = 10  
    print(f"Local variable: {local_var}")  
  
# Run the function  
local_variable_example()
```

Challenge 4: Define a function that declares a local variable and prints it

```
# Complete the code below
# def print_local():
#     ...
```

Slide 10: Global Variable

```
global_var = 20

def global_variable_example():
    print(f"Global variable: {global_var}")

# Run the function
global_variable_example()
```

Challenge 5: Modify the global variable within a function

```
# Complete the code below
# def modify_global():
#     global global_var
#     ...
```

Slide 11-13: Passing Arguments to Functions

```
# Example 6: Passing Multiple Arguments
def print_full_name(first_name, last_name):
    print(f"Full name: {first_name} {last_name}")

# Run the function
print_full_name("John", "Doe")
```

Challenge 6: Define a function that takes three arguments and prints them

```
# Complete the code below
# def print_three(a, b, c):
#     ...
```

Slide 14-15: Keyword Arguments

```
def describe_pet(animal_type, pet_name):
    print(f"\nI have a {animal_type}.")
    print(f"My {animal_type}'s name is {pet_name}.")

# Run the function using keyword arguments
describe_pet(animal_type='hamster', pet_name='Harry')
```

Challenge 7: Define a function with keyword arguments and call it

```
# Complete the code below
# def show_vehicle(type, brand):
#     ...
```

Slide 10-12: Default Argument Values

```
# Example 8: Default Argument
def print_greeting(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

# Run the function with and without the default argument
print_greeting("Alice")
print_greeting("Bob", "Hi")
```

Challenge 8: Define a function with a default argument and call it

```
# Complete the code below
# def introduce(name, age=25):
#     ...
```

Slide 13: Value-Returning Functions

```
# Example 9: Value-Returning Function
```

```
def square(number):
    return number * number

# Run the function and print the result
result = square(4)
print(f"The square of 4 is: {result}")
```

Challenge 9: Define a function that returns the cube of a number

```
# Complete the code below
# def cube(number):
#     ...
```

Slides 14-16: The random and math Modules

```
import random
# Generate a random float in the range [0.0, 1.0)
number = random.random()
print("Random float between 0.0 and 1.0:", number)
```

```
import random
# Generate a random integer between 1 and 100
number = random.randint(1, 100)
print("Random number between 1 and 100:", number)
```

```
import math

# Example 10: Using the math Module
def calculate_circle_area(radius):
    return math.pi * radius * radius

# Run the function and print the result
area = calculate_circle_area(5)
print(f"The area of the circle is: {area:.2f}")
```

Challenge 10: Define a function that uses the math module to calculate the square root of a number

```
# Complete the code below
# def calculate_sqrt(number):
#     ...
```

Slide 20-21: Storing Functions in Modules

```
# Example 11: Creating and Using Modules
# Create a file named my_module.py with the following content:
# def my_function():
#     print("Hello from my_module!")

# from my_module import my_function
# my_function()

# Using Functions from a Module in Different Files
# Create another file named main.py with the following content:
# from my_module import my_function

# my_function()
```

uncomment 2 lines below

```
##writefile my_module.py
#def my_function():
#    print("Hello from my_module!")
```

```
#from my_module import my_function
#my_function()
```

Challenge 11: Create your own module with at least two functions and import it here

```
# Complete the code below by creating with your functions
# Create a file named custom_module.py with the following content:
# def function_one():
#     print("Function One")
# def function_two():
#     print("Function Two")

# Import and use the functions from custom_module
# from custom_module import function_one, function_two
# function_one()
# function_two()
```

```
# Example 14: Generating Random Numbers
import random

def roll_die():
    return random.randint(1, 6)

# Run the function and print the result
die_roll = roll_die()
print(f"Rolled a die: {die_roll}")
```

Challenge 13: Define a function that generates a random float between two numbers

```
# Complete the code below
# def random_float(min_value, max_value):
#     ...
```

```
# Example 15: Returning Multiple Values
def min_max(numbers):
    return min(numbers), max(numbers)

# Run the function and print the result
min_val, max_val = min_max([1, 2, 3, 4, 5])
print(f"Min: {min_val}, Max: {max_val}")
```

Challenge 14: Define a function that returns the sum, difference, and product of two numbers

```
# Complete the code below
# def calculate(a, b):
#     ...
```

Returning None from a Function

```
def divide(num1, num2):
    if num2 == 0:
        return None
    else:
        return num1 / num2

# Run the function and handle the result
result = divide(10, 0)
if result is None:
    print("Cannot divide by zero!")
else:
    print(f"Result: {result}")
```

Challenge 15: Define a function that returns None if a number is negative

```
# Complete the code below
# def check_positive(number):
#     ...
```

Conditional Execution of the main Function

```
def main():
    print("This is the main function.")

def secondary_function():
    print("This is a secondary function.")

if __name__ == '__main__':
    main()
```

Challenge 16: Define two functions and conditionally execute one based on a variable value

```
# Complete the code below
# def function_one():
#     ...
# def function_two():
#     ...
```

```
%%writefile nasa.py
# apod.py
import requests
from IPython.display import display, Image

def get_apod(api_key):
    response = requests.get(f'https://api.nasa.gov/planetary/apod?api_key={api_key}')
    if response.status_code == 200:
        data = response.json()
        return data['title'], data['url'], data['explanation']
    else:
        return "Could not fetch APOD data", "", ""

def display_apod(api_key):
    title, url, explanation = get_apod(api_key)
    if title == "Could not fetch APOD data":
        print(title)
    else:
        print(f"Title: {title}\nURL: {url}\nExplanation: {explanation}")
        display(Image(url=url, embed=True))
```

```
# Import the apod module
import nasa

# Use the DEMO_KEY provided by NASA
api_key = "DEMO_KEY"

# Call the function to display APOD data and image
nasa.display_apod(api_key)
```

More on Modules in Class Lab